We now turn to the more involved example in Figure 4, which Slice discretizes automatically. This example emphasizes how our discretization procedure operates across higher-order functions, unbounded recursion, and lists.

The program notation **discrete**$(p_0 : e_0, \ldots, p_{n-1} : e_{n-1})$ on line 6 is the finite-support surface form from Equation (Finite-support elaboration). In particular, the program processed by type inference and covered by the formal calculus is its elaboration to an index-valued **discrete** followed by finite comparisons and conditionals.

The program models the following process: We start by randomly sampling 4 intervals (l. 6 and 13), stored as a list of pairs, where the first component is a flag and the second component is a function that determines whether a given point falls within that interval. We then employ unbounded recursion (l. 36) to repeat the following a potentially unbounded number of times according to a geometric distribution: sample a point y from a gaussian (l. 31), and update the flag of every interval to true whenever y falls within that interval (l. 22). The program returns true iff *all* intervals have been hit at least once (l. 50 and 42). Exact inference on this program therefore determines the probability that this infinite stochastic process hits every interval at least once.

Slice *automatically* produces a *provably equivalent* (cf. §6) discrete program (elaborates the surface program and produces the discrete program shown in §A.5). Even though this . The finite-support surface elaboration is semantics-preserving by definition. The broader equivalence between this continuous flagship program and its discretization is the intended consequence of §6, but remains conditional on the open weak simulation obligation. In particular, the currently discharged strict finite comparison fragment does not cover this program's Gaussian sampling, lists, and recursion. Even though the discrete program still contains unbounded recursion, our back-end automatically determines that its stochastic execution behavior can be modeled as a *finite-state* Markov chain. Our key insight is that exact inference can then be performed via *probabilistic model checking*: we We use Storm [16], which automatically computes that the sought-after probability is 0.034474... for that generated model. See §7.1 for details. To the best of our knowledge, our pipeline is the first capable of performing exact inference on suitable programs containing continuous sampling, higher-order functions, and unbounded recursion. In §7.1.2, we extend this program by conditioning *inside the recursion*.

3 The Slice Language and Type System

We now describe the language (§3.1), its types (§3.2), and the typing rules for discretization (§3.3). Henceforth, we use the terms *expression*, *subexpression*, and *program* somewhat interchangeably when the intended meaning is clear from context.

3.1 The Slice Language

The syntax in Figure 5 extends a standard higher-order functional core with probabilistic choices. Beyond constants, variables, comparisons, Boolean operators, conditionals, pairs, and lists, Slice offers first-class functions, recursion, and observations. The construct **observe**(e) conditions on e holding. Finite-domain constants $k_{\#n}$ range over $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$ as discussed below. For random sampling, we include the usual discrete distributions and any continuous distribution with a tractable cumulative distribution function (one- or two-argument variants such as exponential, laplace, uniform, gaussian, beta, gamma). Adding further distributions is straightforward as long as their CDFs are known; §C lists the ones implemented.

The expression **discrete**$(p_0, \ldots, p_{n-1})$ requires a nonempty probability vector: $n \geq 1$, every $p_i \geq 0$, and $\sum_i p_i = 1$. It returns $k_{\#n}$ with probability p_k.

These are ideal real-valued premises. The implementation separately validates finite binary64 weights. It normalizes compiler-generated weights only within a tight allowance for accumulated CDF arithmetic, and parses source vectors within a documented 10^{-4} rounded-input allowance. These engineering policies do not replace the exact premise above: binary64 evaluation and normalization do not by themselves establish exact preservation of ideal real-valued probabilities, which requires a separate numerical error analysis.

Listings also use the derived surface form **discrete**$(p_0 : e_0, \ldots, p_{n-1} : e_{n-1})$, which samples expression e_i with probability p_i. This form does not extend the formal core. For a fresh variable z, it elaborates before type

$$
\begin{array}{rl}
e ::= x \mid c \mid \text{true} \mid \text{false} \mid k_{\#n} \mid () & \text{constants} \\
\mid \textbf{let } x = e_1 \textbf{ in } e_2 & \text{let binding} \\
\mid \textbf{uniform}(e_1, e_2) \mid \textbf{gaussian}(e_1, e_2) \mid \textbf{exponential}(e) \mid \ldots & \text{continuous distributions} \\
\mid \textbf{discrete}(p_0, \ldots, p_{n-1}) & \text{discrete distribution} \\
\mid \textbf{discrete}(p_0 : e_0, \ldots, p_{n-1} : e_{n-1}) & \text{surface finite-support distribution} \\
\mid e_1 < e_2 \mid e_1 \leq e_2 \mid e_1 <_{\#n} e_2 \mid e_1 \leq_{\#n} e_2 & \text{comparisons} \\
\mid e_1 \;\&\&\; e_2 \mid e_1 \;||\; e_2 \mid \text{not } e & \text{Boolean operations} \\
\mid \textbf{if } e_1 \textbf{ then } e_2 \textbf{ else } e_3 & \text{conditional} \\
\mid (e_1, e_2) \mid \textbf{fst } e \mid \textbf{snd } e & \text{pairs} \\
\mid \textbf{fun } x \;\to\; e \mid e_1\; e_2 \mid \textbf{fix } f\; x := e & \text{functions} \\
\mid \textbf{observe } e \mid e_1; e_2 & \text{effects} \\
\mid \text{nil} \mid e_1 :: e_2 \mid (\textbf{match } e \textbf{ with } \mid \text{nil} \to e_{\text{nil}} \mid e_1 :: e_2 \to e_{\text{cons}} \textbf{ end}) & \text{lists} \\
\mid \textbf{diverge} & \text{diverge}
\end{array}
$$

Fig. 5. Syntax Core and derived surface syntax of the Slice language. Slice supports a variety of continuous distributions with one and two parameters; for a full list of supported distributions, see Appendix C.

inference as

$$
\begin{aligned}
&\text{elab}\big(\textbf{discrete}(p_0 : e_0, \ldots, p_{n-1} : e_{n-1})\big) \\
&\quad = \textbf{let } z = \textbf{discrete}(p_0, \ldots, p_{n-1}) \textbf{ in } \textbf{if } z <_{\#n} 1_{\#n} \textbf{ then } \text{elab}(e_0) \textbf{ else } \cdots \\
&\qquad\qquad \textbf{if } z <_{\#n} n - 1_{\#n} \textbf{ then } \text{elab}(e_{n-2}) \textbf{ else } \text{elab}(e_{n-1}).
\end{aligned}
$$

(Finite-support elaboration)

For $n = 1$, the body is just $\text{elab}(e_0)$. This notation is defined only when each $p_i \in [0, 1]$ and $\sum_i p_i = 1$. Whenever every e_i has type τ, the elaborated expression has type τ by Discrete, FinLess, If, and Let. Only the selected branch is evaluated. We assign the surface form the semantics of this elaboration. Thus the conditional theorem below concerns the resulting core expression; it is not a separate theorem about an additional operational-semantic case. The implementation validates the weights, performs this capture-avoiding elaboration in the parser, and sends the resulting core term through the same type inference and discretization passes.

3.2 Types

We formalize the types that Slice supports. A *value* set V is a finite set of reals or $\top$. Elements of $\{< x \mid x \in \mathbb{R}\} \cup \{\leq x \mid x \in \mathbb{R}\}$ are called *cuts*. Cut sets induce intervals partitioning the reals as follows: Given a non-empty cut set $B = \{\sim_1 x_1, \ldots, \sim_n x_n\}$ with $x_1 \leq \ldots \leq x_n$, we define the set $\text{Intervals}(B)$ of intervals it induces as expected, e.g., $\text{Intervals}(\{<0.1, \leq 0.5\}) = \{(-\infty, 0.1), [0.1, 0.5], (0.5, \infty)\}$. The strict and non-strict cuts differ only in which side contains the boundary point: $\text{Intervals}(\{<c\}) = \{(-\infty, c), [c, \infty)\}$, whereas $\text{Intervals}(\{\leq c\}) = \{(-\infty, c], (c, \infty)\}$. Thus a boundary generated by $e < c$ places c on the right, while a boundary generated by $e \leq c$ places c on the left. Moreover, we let $\text{Intervals}(\emptyset) = \{(-\infty, \infty)\}$ and $\text{Intervals}(\top) = 2^{\mathbb{R}}$. We denote by $\text{Intervals}(B)_i$ the i-th interval of $\text{Intervals}(B)$, assuming $\text{Intervals}(B)$ is 0-indexed, i.e., $i \in \{0, ..., n\}$. We denote the set of all cut- and value sets, respectively, by $\mathcal{B}$ and $\mathcal{V}$.

Definition 3.1 (Types). For $n \in \mathbb{N}$, $B \in \mathcal{B}$, and $V \in \mathcal{V}$, types τ adhere to the grammar

$$
\tau ::= \textbf{int} \mid \textbf{bool} \mid \textbf{unit} \mid \textbf{fin}(n) \mid \textbf{float}[B; V] \mid \tau_1 * \tau_2 \mid \tau_1 \to \tau_2 \mid \textbf{list}(\tau)\,.
$$

Standard types cover integers, booleans, ordered pairs, functions, and lists. Finite types $\textbf{fin}(n)$ contain $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$; the $\#n$-subscript reminds inference backends such as Dice that these values live in a finite domain. The float type $\textbf{float}[B; V]$ annotates a real-valued expression with its cut set B (an over-approximation of the comparisons it participates in) and value set V (an over-approximation of the values it can take). $B = \top$ means the expression may be compared to arbitrary reals; $B = \emptyset$ means it will never be compared; $V = \top$ indicates an unrestricted result.

$$\mathcal{D}[\![\tau]\!] = \begin{cases} \textbf{fin}(|B|+1) & \text{if } \tau = \textbf{float}[B;V] \text{ and } B \neq \top \\ \textbf{float}[B;V] & \text{if } \tau = \textbf{float}[B;V] \text{ and } B = \top \\ \text{(straightforward recursion on type structure)} & \text{otherwise} \end{cases}$$

Fig. 14. Transformation of types under $\mathcal{D}$, where $\mathcal{D}$ acts on types.

where $B = \{\leq 2.5, \leq 3.5\}$, and where e_1 : `float[B; T]` and e_2 : `float[B; {2.5,3.5}]`. Intuitively, e_1 samples from a mixture of a uniform and a dirac distribution, and e_2 samples from a finite distribution. Why can $e_1 \leq e_2$ be discretized even though neither e_1 nor e_2 is a constant? As suggested by its type, e_2 can take only finitely many values, namely $\{2.5, 3.5\}$. We apply $\mathcal{D}[\![\cdot]\!]$:

```
(if discrete(0.5, 0.5) <#2 1#2 then discrete(0.25, 0.1, 0.65) else 0#3)
<=
(if discrete(0.5, 0.5) <#2 1#2 then 0#3 else 1#3)
```

Notice that the probability of e_1 evaluating to some value in $(-\infty, 2.5]$ indeed *coincides* with the probability of the discretization of e_1 evaluating to $0_{\#3}$. For e_2, on the other hand, the probability of it evaluating to 2.5 (resp. 3.5) *coincides* with the probability of its discretization evaluating to $0_{\#3}$ (resp. $1_{\#3}$). For this reason, we obtain a semantically equivalent discretization of $e_1 \leq e_2$.

5.0.1 Discretization of Types. We conclude by extending $\mathcal{D}[\![\cdot]\!]$ to types. Figure 14 shows how type τ is discretized to type $\mathcal{D}[\![\tau]\!]$. We then obtain the following natural result, proved in §E.1:

Theorem 5.4 (Discretization Is Well-Typed). *Let* $\Gamma \vdash e : \tau$. *We have* $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, *where* $\mathcal{D}[\![\Gamma]\!]$ *transforms all types in* Γ, *i.e.,* $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ *for all* $x \in \mathrm{dom}(\Gamma)$.

6 Operational Semantics and Soundness

We now establish the soundness The intended soundness property of our approach in the following sense: For *every* is that for every core expression e : **bool**, possibly including *continuous sampling, higher-order functions, recursion, and conditioning,* continuous sampling, higher-order functions, recursion, and conditioning,

e : **bool** *and* $\mathcal{D}[\![e : \textbf{bool}]\!]$: **bool** *(obtained from Figure 13) are semantically equivalent.*

The weighted finite-support notation used in Figure 4 is assigned the semantics of its elaborated core expression from Equation (Finite-support elaboration). The theorem therefore applies after elaboration; it does not add a surface-syntax case to the operational semantics or prove a separate correspondence with some independently defined surface semantics. The first step is to make formal sense of semantic equivalence. For that, we will define a monadic operational semantics for our language in §6.1. We will then proceed by proving the above claim. This is non-trivial: discretization The currently discharged proof covers the stated strict finite comparison fragment. Extending it to every core expression requires a weak simulation obligation; that obligation remains open. Discretization decisions are driven by non-local, type-propagated information, so we cannot proceed the missing argument cannot be replaced by a local rule-by-rule argument. To prove the claim rigorously, we relate the executions of e and $\mathcal{D}[\![e]\!]$ through a coupling-style logical relation in §6.1. proof.

6.1 Operational Semantics

The type-directedness of our discretization function $\mathcal{D}[\![\cdot]\!]$ requires our soundness argument to depend on the types of the given (sub-)expressions as well. We thus start by defining a *type-dependent semantic functional*, i.e., we define for every type τ a function $\langle\cdot\rangle : Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$, mapping each *fully type-annotated* expression $e : \tau$ to a *sub*-probability measure $\langle e : \tau\rangle$ over values of type τ or $\bot$. In the presence of possibly unbounded recursion *and* conditioning, our semantics has to distinguish between non-termination and observation failure, which we realize as follows: The "missing" probability mass in $\langle e : \tau\rangle$ is the probability that e diverges. The probability of $\bot$ is the probability of encountering an observation failure. With this model,

6.2 Soundness via Probabilistic Couplings

We first state the central theorem, then outline our coupling-style correctness argument.

Theorem 6.1 (Conditional soundness). *Let e : **bool**. Then Assume the weak simulation obligation for the source and discretized operational semantics. For every core expression e : **bool** in its stated fragment,* $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$.

Intuitively, we prove that the two programs can execute in a synchronized manner such that their probabilities of terminating in value v coincide for each $v \in \{\text{true}, \text{false}, \bot\}$.

Towards this end, define for every type τ the set $P_\tau = \{(e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]) \mid e : \tau\}$. We define a *coupled small-step semantics* $[\![\cdot, \cdot]\!] \colon P_\tau \to \mathcal{D}(P_\tau + \bot)$ (cf. Tables 3 and 4), which models the simultaneous execution of e and $\mathcal{D}[\![e : \tau]\!]$. The key property of this coupled semantics is that the semantics of both e and $\mathcal{D}[\![e : \tau]\!]$ can be recovered in a sense we formalize next.

Given $\mu \in \mathcal{D}(P_\tau + \bot)$, we define the *projection of μ onto the i-th component* for $i \in \{1, 2\}$ as

$$\pi_i(\mu) = \lambda A. \int_{x \in P_\tau + \bot} \begin{cases} \delta_{e_i}(A) & x = (e_1, e_2) \\ \delta_\bot(A) & x = \bot \end{cases} \mu(dx)\,,$$

With projections, we recover the semantics of e and $\mathcal{D}[\![e : \tau]\!]$ as follows:

Lemma 6.2. *Let $e : \tau$. We have:*

(1) $\pi_1([\![e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![e : \tau]\!]$.
(2) $\pi_2([\![e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![\mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]$.

Now, using a limit construction analogous to §6.1.2 for obtaining a coupled *big-step* semantics $\langle \cdot, \cdot \rangle \colon P_\tau \to \mathcal{D}(V_\tau + \bot)$, where $V_\tau = \{(v : \tau, \mathcal{D}[\![v]\!] : \mathcal{D}[\![\tau]\!]) \mid v : \tau \text{ is a value}\}$, we prove a suitable continuity property of projection (cf. Lemma F.51) to conclude that Lemma 6.2 transfers to $\langle \cdot, \cdot \rangle$. Finally, observing that $V_{\mathbf{bool}} + \bot = \{(\text{true}, \text{true}), (\text{false}, \text{false}), \bot\}$ yields the desired claim:

Proof of Theorem 6.1. Let $v \in \{\text{true}, \text{false}, \bot\}$. We have

$$\begin{aligned}
&\langle e : \mathbf{bool}\rangle(v) \\
&= \pi_1(\langle e : \mathbf{bool}, \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle)(v) && \text{(Lemma 6.2.1 for } \langle \cdot, \cdot \rangle) \\
&= \pi_1(\langle e : \mathbf{bool}, \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle)(v) && (P_{\mathbf{bool}} \text{ is discrete}) \\
&= \langle e : \mathbf{bool}, \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle((\text{true}, \text{true})) \cdot \delta_{\text{true}}(v) \\
&\quad + \langle e : \mathbf{bool}, \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle((\text{false}, \text{false})) \cdot \delta_{\text{false}}(v) \\
&\quad + \langle e : \mathbf{bool}, \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle(\bot) \cdot \delta_\bot(v) && \text{(definition of } \pi_1\text{, argument is discrete)} \\
&= \pi_2(\langle e : \mathbf{bool}, \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle)(v) && \text{(definition of } \pi_2\text{, argument is discrete)} \\
&= \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle(v)\,. && \text{(Lemma 6.2.2 for } \langle \cdot, \cdot \rangle)
\end{aligned}$$

7 Implementation, Case Studies, and Evaluation

We have implemented the Slice type inference and discretization procedures in OCaml. The implementation is open-source and available at https://github.com/[anonymized], together with all benchmarks, scripts, and a Docker image for reproduction. To compute the interval masses of continuous distributions, we use the OCaml GSL package [19] for cumulative distribution function calculations. All experiments were run in this Docker image on an Apple M3 with 16 GB of RAM.

This section is driven by two research questions:

(1) Does Slice enable exact inference on programs combining continuous sampling, higher-order functions, and unbounded recursion?
(2) On programs that are within the scope of existing tools for exact inference, is the combination of Slice and discrete inference back-ends competitive?